

Glazyr's Required Data and Extension Feasibility

The Glazyr assistant likely needs to monitor the user's browsing context to feed into its AI. This includes **active tab URLs and navigation events**, the **page DOM and user interactions**, form inputs or clicks, and possibly **network requests** for page resources. A modern Chrome extension can capture much of this via its APIs. For example, an extension's background script can listen to `chrome.tabs.onUpdated` and `chrome.webNavigation` events to detect page loads and URL changes (with the `"tabs"` or `"webNavigation"` permission) ¹. Content scripts injected into each page (via the `content_scripts` manifest or programmatically) can access the **page's DOM**, read text, and attach handlers for form inputs or clicks ². In particular, content scripts "run in the context of web pages... [using the] DOM, they are able to read details of the web pages... make changes to them, and pass information to their parent extension" ². Thus a content script could, for example, `document.querySelector` the page text or listen for `input` events on forms and send that data back to the background via `chrome.runtime.sendMessage()`. For network traffic, the extension's background can use `chrome.webRequest` (with `"webRequest"` and host permissions) to observe or even block/modify HTTP requests ³ ⁴. (In MV3, blocking is limited, but observing request metadata is still allowed.) In short, most browsing behavior—URLs, page content, DOM events, cookies or storage, screenshots (`tabs.captureVisibleTab`), console logs, etc.—is accessible through extension APIs ¹ ² ³. Only very low-level or cross-domain OS-level data (like encryptions, certificates, or outside-browser traffic) would require a Chromium fork or external tooling.

- **Browsing/navigation:** `chrome.tabs` and `chrome.webNavigation` let the extension know when a user switches or loads pages and what the URL is ¹. With the `"tabs"` permission the extension can even capture a screenshot or tab language ¹.
- **DOM interaction and form input:** Content scripts can inspect any part of the page DOM and listen for user actions (clicks, keystrokes, form submits). They run in an isolated world but can manipulate page elements and send data to the extension ². For example, a script could record which input fields changed or capture `window.location` and page text.
- **Network traffic:** The extension can declare `chrome.webRequest` listeners (e.g. `onBeforeRequest`, `onCompleted`) to see outgoing and incoming requests on pages where it has host permissions ³ ⁴. This allows it to observe URLs of subresources (scripts, XHR, etc.), though note that in Manifest V3 it can no longer synchronously block or modify them except via `declarativeNetRequest`.
- **Cookies, storage, and other data:** The extension can use `chrome.cookies`, `chrome.storage`, or even the content script's `document.cookie` to capture session data as needed. Any data stored by the page (localStorage, IndexedDB) can often be read by content scripts unless sandboxed.

Thus, a Chrome extension *can* access virtually all high-level browsing activity Glazyr needs (current page, DOM content, user inputs, network calls) as long as the user grants the proper permissions.

Extension APIs vs Chromium Fork

A full Chromium fork (like Brave or a custom browser) can instrument *everything* – modify the browser UI, hook into the network stack, integrate native code (e.g. embedded AI models), and operate below the extension sandbox. A fork can install new UI panels, open tabs programmatically without permission prompts, and capture any data (even HTTPS payloads if modified at the network layer). For example, BrowserOS – a privacy-focused agentic browser built as a Chromium fork – natively integrates local LLMs and a side panel UI because it controls the entire browser stack ⁵. By contrast, an extension is sandboxed: it *cannot* arbitrarily modify Chrome's internal code or UI outside the extension APIs. Extensions rely on declared permissions and cannot, for example, bypass Chrome's same-origin policies or inject into `chrome://` pages.

Capabilities of an Extension: Within the web context, an extension is surprisingly powerful. As seen with the open-source MCP Chrome extension (Hangwin's mcp-chrome), a simple extension can “expose your Chrome browser functionality to AI assistants” ⁶. Using content scripts and background scripts, it can fully automate browsing tasks (clicks, form fills, navigation) and gather page state. Chrome's APIs let the extension control tabs, capture screenshots, listen to console logs, and modify web pages. Extensions are also cross-browser (with minor changes) and update automatically via the Web Store.

Limitations vs Fork: However, extensions can only use the official Chrome APIs. They cannot, for instance, hook into arbitrary TCP streams or the browser's internal network stack beyond `webRequest`. They cannot modify the Chrome UI (other than their own toolbar icons or popups) or run with native code privileges. A Chromium fork can embed, say, a Python or C++ module to handle network data or run a local LLM directly. Extensions, on the other hand, would rely on web requests or native messaging to external processes if they need heavy lifting. Furthermore, Manifest V3 has tightened restrictions: blocking or altering requests dynamically now requires pre-defined rules (`declarativeNetRequest`) ⁷, and background scripts are service workers (event-driven, not always persistent).

In summary, an extension can cover the core use case of capturing browsing context and DOM events without modifying Chrome itself. For example, Hangwin's MCP server proves an extension can drive complex automation ⁶. But a Chromium fork can do *more*: deep integration (native LLMs, UI changes, advanced privacy hooks) at the cost of requiring users to install a custom browser. Whether Glazyr needs those extra powers depends on its goals. If it only needs page text, clicks, and limited network info, an extension is viable and much easier to deploy. If it wanted to run fully offline LLMs or override browser security, a fork might be chosen (as with BrowserOS ⁵).

User Experience and Permissions

As a browser extension, Glazyr would present itself as a Chrome (or cross-browser) addon. On installation, Chrome will display its requested permissions. For example, if Glazyr needs to see all websites, it would list something like “Read and change all your data on the websites you visit” (coming from host permissions like `<all_urls>`) and “Manage your downloads” or “Capture visible tabs” if needed. This is inevitable for an extension that passively monitors browsing. Chrome prompts the user to approve these on install (or on first use, if using `activeTab`). After permission is granted, the extension can run continuously (as a service-worker background script in MV3) and inject content scripts into pages matching its host list.

Passive Tracking: Once installed, Glazyr could quietly run in the background. It might listen to tab updates and record the user's active URL and title (with the `"tabs"` permission ¹). Content scripts could automatically scrape page text as soon as a page finishes loading. All of this would be transparent to the user except for Chrome's initial permission UI. However, the extension's icon (in the toolbar) could indicate activity (e.g. change color when active on a page) or show notifications if something needs attention.

Interactive UI: Glazyr would likely provide a UI for the user to ask questions or see results. This could be implemented as a popup window when the toolbar icon is clicked, a separate browser tab, or even an injected sidebar on the web page (Chrome now supports a side panel API ⁸). For example, clicking Glazyr's icon might open a chat UI (perhaps the same codebase seen in the `glazyr` repo) where the user can type queries about the current page. Alternatively, Glazyr might add a context-menu item ("Ask Glazyr") to let the user manually invoke analysis on selected text. In any case, interaction with the tool would be optional: the extension could work passively in the background and only pop up the UI when the user needs it.

Permission Prompts: In Manifest V3, each time Glazyr first tries to use a powerful API or access new hosts, Chrome can prompt the user. For example, if Glazyr tries to capture a screenshot of the tab, it needs the `"activeTab"` or `"tabs"` permission, which is an extra prompt. If it tries to intercept network on a host not yet granted, Chrome will ask for access to that host. Good practice is to minimize these prompts: use `"activeTab"` to get temporary access when the user clicks the icon, or only request broad `"<all_urls>"` if truly needed. The user can always revoke permissions or disable the extension later.

In short, users would install Glazyr like any extension. They would see clear warnings about data access, then enjoy a passive assistant that can read the page and respond via its UI. Typical interactions might involve clicking the extension icon to open Glazyr's chat panel, or otherwise the assistant working in the background ("tracking mode") and displaying an icon badge or notification when new context is captured.

Architecture Recommendations for an Extension

A clean extension-based design would separate *capture*, *processing*, and *UI*:

- **Background Service Worker:** The extension's service worker (background script) manages global state and communication. It can listen for events like `chrome.tabs.onActivated` or `chrome.webNavigation.onCompleted` to know which URL is active. It might maintain an in-memory or `chrome.storage.local` record of recent pages or interactions. It also handles messaging: receiving data from content scripts and forwarding it to the AI logic (or to a local/remote server).
- **Content Scripts:** One or more scripts are injected into matching pages (via the `"content_scripts"` key in `manifest.json` ⁹ or via `chrome.scripting.executeScript`). These scripts read the DOM (for example, `document.body.innerText`, metadata, links) and detect user events (clicks, keypresses, form submissions). For each relevant action (say, a form submit), the content script sends a message to the background with details. This decouples UI from data collection. Content scripts can also dynamically inject UI elements if needed (e.g. an on-page "Ask Glazyr" button), though care must be taken to avoid conflicting with the site's code (use isolated DOM or shadow DOM).

- **MCP/Server Component (optional):** If Glazyr uses a Model Context Protocol server like YetiBrowser, the extension would connect to a local MCP server (e.g. via WebSockets to `localhost`). The background script could forward captured data to this server and receive back AI-generated actions or summaries. For example, on each page load the background might send a “browser_snapshot” command to the MCP server and relay the JSON result to the content script/UI. The YetiBrowser model shows a robust pattern: a local Node MCP server and a Chrome extension talking over WebSockets ¹⁰. Glazyr could use a similar approach if it needs complex queries or automation. If no server is used, the extension’s background could itself call a cloud AI API (via `fetch` with appropriate CORS permissions).
- **User Interface:** The UI (chat window) can be built as an extension **popup** (declared in `manifest.json`) or as a full page in a new tab. It might even be a content script-injected panel or a [Chrome side panel](#) ⁸. The UI would communicate with the background/service worker to send user questions and display answers. For example, a popup React app could use `chrome.runtime.sendMessage` to request the current page context from the background, then display the AI’s answer. Chrome’s `chrome.storage` or messaging APIs can link the background and UI.
- **Permissions Setup:** In `manifest.json`, Glazyr would request `"permissions": ["tabs", "storage", "activeTab"]` and `"host_permissions": ["<all_urls>"]` if needed. As the [Tabs API docs](#) describe, using `"tabs"` lets the extension query tab URLs and capture visible tab content. The `"activeTab"` permission can be used to get temporary access when the user clicks the icon (to reduce permanent privileges). All injection scripts would be white-listed via `"matches": ["<all_urls>"]` or a subset in the `content_scripts` section ⁹.

Example Workflow: When the user navigates to a page, the background notes the URL change. It injects (or has injected) a content script that scrapes the page text and sends it back via `chrome.runtime.sendMessage`. The background might then call an AI (locally or remote) with that text. When the user clicks Glazyr’s button, the extension opens its chat UI and shows the AI response. If the user asks Glazyr about page details (“What is this page about?”), the extension can highlight text or use the current DOM to formulate an answer.

Existing projects demonstrate this pattern. Hangwin’s Chrome MCP server uses a similar extension architecture to give AI assistants full browser context ⁶. YetiBrowser MCP shows how an extension plus a local server can be “local-first” and fully auditable ¹⁰. In Glazyr’s case, a standalone extension (possibly paired with a light backend) should suffice. The main recommendation is to keep the extension code modular: content scripts do page-level grabs, the background handles coordination and permissions, and a dedicated UI component handles user interaction.

References: Official Chrome extension docs detail these APIs (tabs, webRequest, content scripts) ¹ ³ ². The Hangwin MCP and YetiBrowser projects are concrete examples of extension-based browser assistants ⁶ ¹⁰, while projects like BrowserOS show the alternative Chromium-fork approach ⁵. Using these patterns, Glazyr can likely be implemented entirely as an installable extension with only minimal loss of capability compared to a custom browser.

1 chrome.tabs | API | Chrome for Developers

<https://developer.chrome.com/docs/extensions/reference/api/tabs>

2 Content scripts | Chrome for Developers

<https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts>

3 4 7 chrome.webRequest | API | Chrome for Developers

<https://developer.chrome.com/docs/extensions/reference/api/webRequest>

5 BrowserOS: The Open Source Agentic Browser Reimagining Web Interaction | FunBlocks AI Reviews

<https://www.funblocks.net/aitools/reviews/browseros-2>

6 GitHub - hangwin/mcp-chrome: Chrome MCP Server is a Chrome extension-based Model Context Protocol (MCP) server that exposes your Chrome browser functionality to AI assistants like Claude, enabling complex browser automation, content analysis, and semantic search.

<https://github.com/hangwin/mcp-chrome>

8 chrome.sidePanel | API - Chrome for Developers

<https://developer.chrome.com/docs/extensions/reference/api/sidePanel>

9 Manifest - content scripts | Chrome Extensions | Chrome for Developers

<https://developer.chrome.com/docs/extensions/reference/manifest/content-scripts>

10 GitHub - yetidevworks/yetibrowser-mcp

<https://github.com/yetidevworks/yetibrowser-mcp>